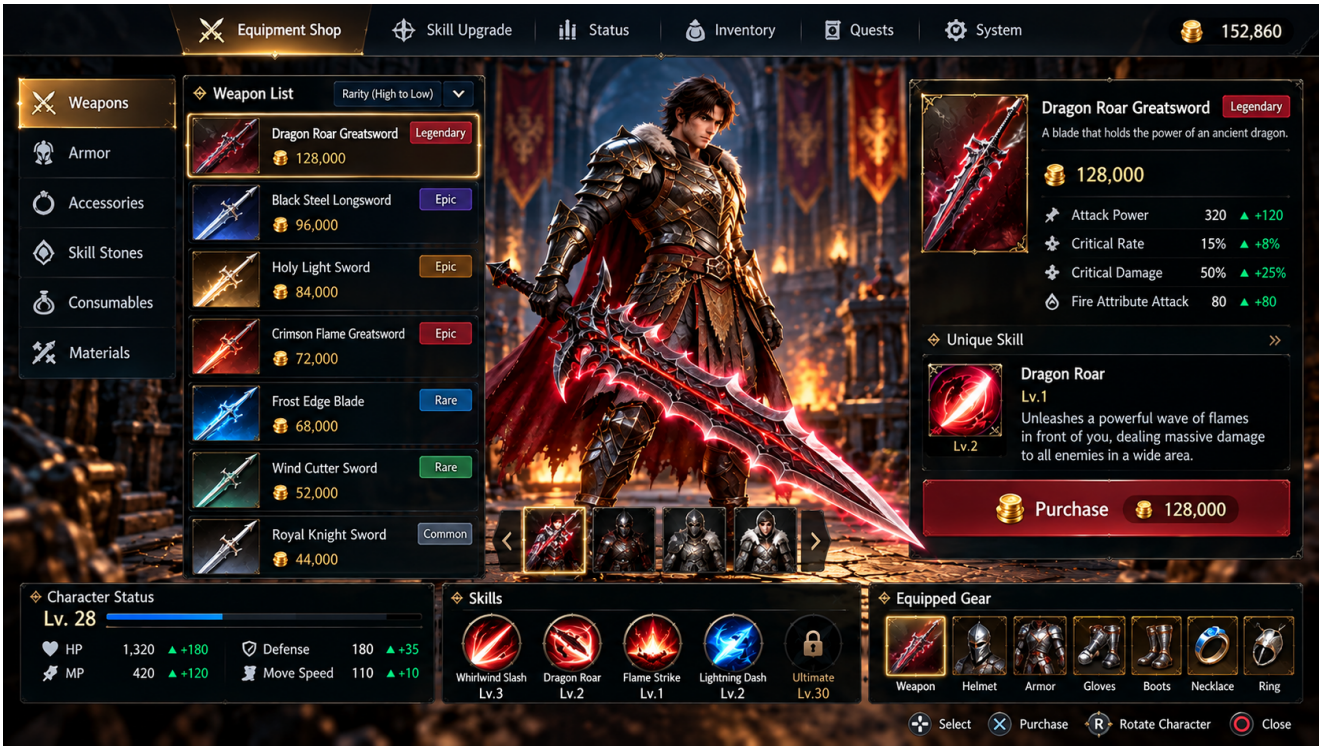
UNITY 3D FANTASY RPG

Equipment, Character Customization & Progression System

A showcase concept for an action RPG interface that brings together a cinematic 3D character, equipment purchasing, attribute comparison, skill progression, and inventory management.



Target engine	Core systems	Presentation
Unity · C#	Equipment, skills, stats, inventory	Cinematic 3D + RPG interface

Showcase scope: This document presents the visual concept and a practical Unity implementation outline. Only features that have been implemented and tested in a playable Unity build should be represented as completed functionality.

1. Project Overview

The concept represents an equipment and character progression screen for a dark fantasy action RPG. The player can browse equipment categories, compare item attributes, review a special skill, and see character status without leaving the main interface. A large central character preview anchors the experience and is intended to reflect changes in equipped weapons and armor.

Primary design objectives

- **Immediate character feedback:** keep the warrior large and central so equipment changes remain visually meaningful.
- **Fast item comparison:** present item rarity, price, core stats, and skill effects in a consistent detail panel.
- **Unified equipment workflow:** group weapons, armor, accessories, skill stones, consumables, and materials into clear categories.
- **Visible progression:** surface level, health, mana, defense, movement speed, and ability levels.
- **Premium game presentation:** combine dark metal UI panels, gold accents, dramatic forge lighting, and detailed 3D assets.

Core player journey

Browse	Inspect	Compare	Equip
Choose a category and item.	Review price, rarity, stats, and skill effects.	Check the expected change to character attributes.	Purchase or equip and update the character preview.

2. Interface Breakdown

The layout is divided into stable regions so the player can understand the system at a glance and compare items without losing context. The exact behavior depends on the final Unity scene, prefabs, data model, and input setup.

Top navigation and currency

Navigation tabs lead to Equipment Shop, Skill Upgrade, Status, Inventory, Quests, and System. A currency display remains visible so players can judge affordability before purchasing.

Left category rail

Weapons, Armor, Accessories, Skill Stones, Consumables, and Materials form the main item categories. The selected category should be clearly highlighted and navigable by mouse, controller, or touch as required.

Item catalog

Each item row contains an icon, name, rarity, and price. Sorting by rarity or price helps players find relevant equipment quickly.

Central 3D character preview

The warrior occupies the visual center. Weapon and armor changes should update the character's equipped mesh, attachment points, materials, and preview pose as applicable.

Item detail panel

The selected item is shown with its name, rarity, price, attack and defense values, critical attributes, elemental bonus, and any unique skill description.

Purchase action

The purchase control should validate currency, ownership, item eligibility, and inventory rules before updating the player's balance and equipment state.

Character status

Level, HP, MP, defense, and movement speed summarize the character's current state. Before/after deltas can help communicate the impact of a proposed item.

Skills and equipped gear

Skill icons show ability names and levels, while the equipped-gear row summarizes the weapon, helmet, armor, gloves, boots, necklace, and ring.

3. Equipment & Inventory System

A maintainable equipment system should represent each item as structured data rather than hard-coding values into UI elements. This makes it easier to add new weapons, balance item statistics, and reuse the same interface across categories.

Suggested item data

- **Identity:** unique ID, display name, description, category, icon, and 3D model reference.
- **Rarity:** common, rare, epic, legendary, or a project-specific tier.
- **Economy:** purchase price, sell value if supported, and any unlock requirements.
- **Attributes:** attack, defense, critical rate, critical damage, elemental damage, and other relevant modifiers.
- **Equipment rules:** valid slot, character class restrictions, minimum level, and stackability.
- **Visual references:** model or prefab, material variants, attachment point, and preview animation.

Purchase and equip flow

1. Select	2. Validate	3. Commit	4. Refresh
Player chooses an item.	Check balance, requirements, and inventory capacity.	Deduct currency and update item ownership.	Update stats, equipped slot, and character preview.

Important edge cases

- Insufficient currency should prevent the purchase and explain why.
- Repeated input should not charge the player twice for one purchase.
- Equipment replacement should update the correct slot and recalculate attributes exactly once.
- A failed purchase or invalid item should leave the prior state unchanged.
- If saving is supported, the inventory and equipped state should persist across sessions.

4. Character Preview & Skill Progression

Dynamic character preview

The central preview is the defining interaction in this screen. A practical Unity implementation can use a dedicated character preview camera and lighting rig, with the character displayed in a render texture or a separate preview scene. The character model should update when the selected equipment is applied, while keeping the interface stable.

- Use a consistent skeleton and compatible attachment points for weapons and wearable gear.
- Swap meshes or prefabs for helmets, armor, gloves, boots, and accessories when the corresponding slot changes.
- Attach weapons to the correct hand or back socket and ensure the preview pose supports clear visibility.
- Update materials and visual effects when equipment has elemental or rarity-specific presentation.
- Provide a fallback for missing assets and avoid rebuilding the entire character for minor UI changes.

Skill progression

The skill panel can present active abilities, levels, unlock conditions, and upgrade costs. A skill definition should separate display data from runtime behavior, so balance changes do not require rewriting the UI.

- **Skill definition:** ID, name, icon, description, maximum level, unlock requirements, and upgrade cost.
- **Runtime effect:** damage coefficient, cooldown, range, resource cost, and effect references.
- **Upgrade validation:** confirm prerequisites and available currency or skill points before applying an upgrade.
- **Feedback:** show the new level, updated values, and a clear success or failure state.

5. Recommended Unity Architecture

The following is a recommended architecture for implementing the concept in Unity using C#. It separates data, gameplay rules, character visuals, and presentation to reduce coupling and simplify testing.

Component	Responsibility	Possible Unity approach
Item definitions	Store item metadata and base stats.	ScriptableObject assets.
Inventory service	Track owned items and stack counts.	C# service or domain class.
Equipment service	Manage equipment slots and stat modifiers.	C# service with change events.
Currency / shop	Validate purchases and update balances.	Transaction-style method with validation.
Character visual controller	Apply meshes, materials, and weapon attachments.	MonoBehaviour on preview character.
Skill system	Track skill levels, unlocks, and upgrades.	Skill data assets plus runtime state.
UI presenters	Display item lists, stats, status, and purchase feedback.	Unity UI or UI Toolkit views.
Save system	Persist currency, inventory, and equipment.	Versioned save data and serialization.

Suggested event-driven updates

- OnItemSelected — refresh the detail panel and comparison values.
- OnPurchaseCompleted — update currency, ownership, and purchase feedback.
- OnEquipmentChanged — recalculate stats and refresh the 3D character preview.
- OnSkillUpgraded — update skill level, costs, and related attributes.
- OnCharacterStatsChanged — refresh HP, MP, defense, movement speed, and derived values.

6. Performance, Quality & Validation

Performance considerations

- **Preview rendering:** isolate the character preview camera and tune render texture resolution for the target platform.
- **Asset management:** use appropriate texture sizes, LODs, compression, and Addressables where project scale warrants them.
- **UI efficiency:** avoid rebuilding every item row when only the selected item changes; use pooling for large catalogs.
- **Animation:** reuse the character rig and avoid unnecessary animator or physics work while the shop is open.
- **Profiling:** measure CPU, GPU, memory, and loading behavior with Unity Profiler on target hardware rather than relying on visual impressions.

Functional acceptance checklist

- Selecting a weapon updates the detail panel with the correct name, rarity, price, and attributes.
- A purchase is accepted only when the player meets the requirements and has enough currency.
- Successful purchases update the currency balance and item ownership exactly once.
- Equipping an item updates the correct slot, calculated attributes, and central character appearance.
- Skill upgrades validate costs and prerequisites and display the updated skill level.
- Invalid operations provide understandable feedback without corrupting player state.
- Save/load restores the expected inventory, equipment, currency, and progression when persistence is enabled.
- The interface remains usable at the intended resolution and with the intended input method.

Recommended test coverage

- Unit tests for purchase validation, stat calculations, equipment rules, and skill upgrades.
- Play Mode tests for end-to-end shop interactions and character preview updates.
- Manual testing with mouse/keyboard and controller or touch, depending on target platforms.
- Profiling on representative hardware to validate frame time and memory budgets.

7. Implementation Roadmap

Phase	Work	Deliverable
1. Foundation	Define item data, categories, equipment slots, and stat model.	Validated data structures and initial content.
2. Shop UI	Build navigation, item catalog, detail panel, currency, and purchase feedback.	Functional shop flow.
3. Character preview	Set up preview camera, character rig, attachment points, and gear swaps.	Visible equipment changes.
4. Skills & progression	Implement skill levels, unlock rules, upgrade costs, and stat recalculation.	Connected progression systems.
5. Persistence & QA	Save/load, tests, profiling, accessibility, and input validation.	Release-ready feature set for the agreed scope.

Summary

This Unity-targeted fantasy RPG concept showcases a connected equipment and progression experience: a central 3D warrior, item browsing and purchasing, attribute comparison, gear management, and special-skill upgrades. Its strongest technical focus is the relationship between structured item data, gameplay rules, UI state, and the live character preview. A production build should validate each interaction in Unity and profile the result on the intended target platform.

Portfolio presentation: Use the equipment screen as the visual centerpiece, and pair it with a playable build, short capture, or implementation screenshots if available. Clearly distinguish a visual concept from features that have been implemented and tested.